UCF/GUTT GameTheory: A Proof-Carrying Architecture for Strategic Certification

From Exact-Rational Strategic Specifications to Machine-Checked Causal-Security Guarantees

Michael Fillippini — relationalexistence.com — 2026

Sealed text sha256 6915076b64af1837… — typeset verbatim from the canonical Markdown.

1. Executive Summary

This paper describes a completed, verified system that takes a strategic specification — a repeated game, an enforcement mechanism, a punishment window, a discount factor — and decides, by finite exact-rational computation, whether that specification admits a **formal certificate of causal security**. When a certificate is accepted, the guarantee is a kernel-checked theorem in the Coq proof assistant, with **zero axioms** across the entire proof closure: no history-reactive deviation strategy gains more than ε over compliance against the mediated enforcement machine, for every ε > 0, in sufficiently long play.

The pipeline is: **strategic specification → formal certification → machine-checked causal-security guarantee.** That sentence is the entire public claim, and this paper explains what it means, why it is technically unusual, what it can be used for, and — explicitly — what it does not establish.

Three properties distinguish the system from both conventional game-theoretic analysis and conventional formal verification:

1. **The mathematical chain has no unproved links.** Once a strategic instance is represented in the formal model, the chain from certificate conditions to the stated guarantee contains no unproved axioms and no informal mathematical steps — no numerical approximation and no trusted solver. The external specification-to-Coq translation remains an operational boundary, mitigated by deterministic emission, inspectable generated artifacts, hashing, and cross-machine reproduction.
2. **New structure is data, not development.** Games, enforcement mechanisms, and punishment timing all enter as parameters of one generic theorem family. Certifying a new configuration produces an ~80-line artifact, not a new proof effort.
3. **The architecture was stress-tested against its own failure criteria.** Three structurally distinct compression demonstrations — varying the game, the mechanism, and the timing — were run under frozen criteria (Observations #2 and #3 prospectively preregistered; Observation #1 supplying the baseline), and each demonstrated structural compression rather than case proliferation.

2. The Strategic Assurance Problem

Ordinary game-theoretic analysis can argue that an incentive mechanism deters deviation under stated assumptions. But between the argument and a deployed decision lie several trust gaps:

```
model → mathematical argument → implementation → decision
```

Each arrow can introduce error: the argument may not cover the implemented case; the implementation may not compute what the argument analyzed; the decision may rest on numerical behavior the argument never addressed. In high-stakes settings — protocol incentives, institutional enforcement, automated governance — these gaps are precisely where assurance fails.

The system described here collapses the mathematical portion of that chain into a certification chain whose formal segment contains no unproved or informal transitions (the external specification-to-Coq translation remains an operational boundary; see §9):

```
structured game/mechanism → finite formal obligations
→ executable checker → kernel theorem
```

Every object in that chain is exact: payoffs and enforcement values are rational numbers; the obligations are finitely many inequalities; the checker is a Boolean function mirrored inside the proof development itself; and acceptance discharges the hypotheses of a mechanized theorem.

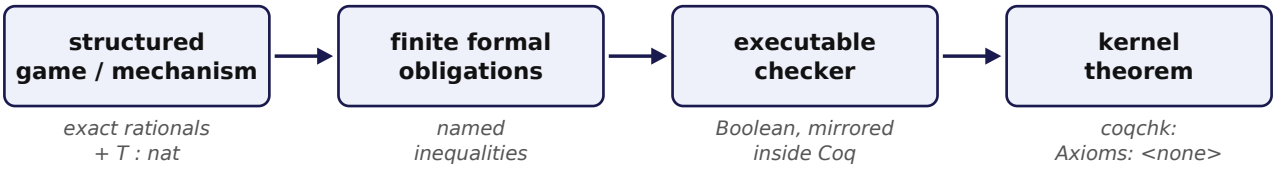

Figure 1 — the certification chain: every arrow is exact.

3. What UCF/GUTT GameTheory Certifies

The certified class is **two-player mediated repeated games with overlay enforcement**. A specification consists of:

- **A game core** G: a 2×2 stage game given by four exact

rational payoff cells per player (cooperative, deviation, sucker, safe).

- **An enforcement mechanism** M (the overlay): reward values

for the compliant party and its partner, a punishment-phase value, a forfeit, a cap M_0 bounding all values, and a punishment window **T** — a natural number, freely chosen.

- **A discount factor** q, an exact rational in [0, 1).

The mediated enforcement machine is an automaton that plays cooperatively, punishes an observed deviator for T rounds, and administers a reward/forfeit phase — the classic trigger-style architecture, mechanized end to end.

Certification is a two-stage decision:

Stage 1 — Admissibility (the static contract). Finitely many inequalities on the specification itself: all values within the cap; the game cells within the cap; the cooperative payoff at least the reward value; and three *reward margins* — 0 < uF_own, v ≤ uF_own, uF_own ≤ uF_partner. These margins were not design choices: they were **discovered** by the formal development — they are the conditions demanded by this certification derivation,

without which the corresponding incentive proof obligations do not close. An inadmissible specification is refused with the failing obligation named; no certificate exists for it at any discount.

Stage 2 — The certificate point. At a chosen (T, q), the executable checker evaluates the dynamic inequalities: a range check, a normal-deviation clause, a reward-deviation clause, and a family of bystander clauses (one per punishment slot, k = 0..T). Acceptance at this stage, together with Stage 1, discharges every hypothesis of the capstone theorem.

The certified guarantee. For every legal history-reactive deviation strategy, every player, and every ε > 0, there exists a sufficiently long horizon beyond which deviation gains no more than ε over compliance against the mediated enforcement machine. What is checked to obtain it: the admissibility contract and the certificate point. Nothing else.

4. A Worked Example

The configuration below is one of the frozen demonstrator examples (and is backed by an actual generated, kernel-checked certificate):

```
Game (both players):
  cooperative = 7/2   deviation = 2
  sucker      = 1     safe      = 2
Mechanism:
  uF_own = 7/2   uF_partner = 5   v = 3/2
  Lf = 5         M0 = 6           T = 3
Discount:  q = 1/2
```

The decision transcript:

```
ADMISSIBILITY (static contract)
  MechanismFeasible        PASS
  MechRewardMargins (×3)   PASS
  GamePayoffFeasible       PASS
  CoopRewardFloor          PASS
CERTIFICATE at q = 1/2
  range 0 ≤ q < 1          PASS
  normal-deviation clause  PASS
  reward-deviation clause  PASS
  bystander clauses k=0..3 PASS
RESULT: ACCEPTED — KERNEL WITNESS: VERIFIED
        (Certificate ID: c8a8da3b4b0879ab)
```

Contrast the two failure modes:

- **Inadmissible.** Set uF_own = 1 (so v = 3/2 > uF_own):

punishment now pays better than cooperation. The static contract refuses at `MechRewardMargins.v_le`, by name. No certificate exists for this mechanism at any discount.

- **Admissible but uncertifiable.** Keep the sound mechanism

but set q = 1/10: the future is discounted too heavily for deterrence. The static contract passes; the reward and bystander clauses fail; the certificate is refused at this point, with the failing inequality shown.

This teaches the system's central distinction:

```
valid model ≠ certifiable strategic point
            ≠ kernel-certified artifact
```

A specification can be structurally sound yet strategically uncertifiable at a given discount; and a decision-surface acceptance is itself distinct from the kernel-checked certificate that formal certification produces.

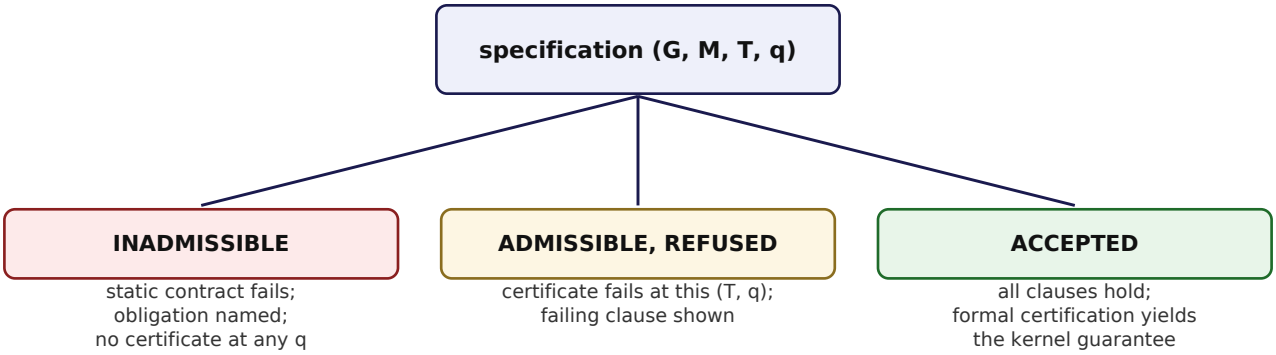

Figure 2 — three outcomes: valid model ≠ certifiable point ≠ certified artifact.

5. Architecture: Specification → Contract → Checker → Theorem

The architecture has four layers, each with an exact interface:

1. **Canonical representation.** Games and mechanisms are records of exact rationals (plus T : nat). The game-dependent information the certificate needs collapses to a two-dimensional sufficient statistic (A, C) — the maximal deviation-side cell and the minimal cooperative cell.
2. **The static contract.** Finitely many inequalities on the representation, each with strategic meaning, each mechanized. The contract is *demanded by the proofs*, not posited: every obligation exists because a theorem refused to close without it.
3. **The executable checker.** A Boolean function over the representation and (T, q), defined inside the proof development, with a mechanized soundness theorem: checker acceptance implies the semantic clauses. The checker is evaluated by reflection (`vm_compute`) inside Coq for generated certificates, and mirrored in exact arithmetic by the public demonstrator.
4. **The capstone theorem.** One theorem consumes the contract and the checker verdict and yields the guarantee, for every window T. Certifying a configuration is one theorem invocation with constant-size witnesses.

A **certifying compiler** (private) turns a JSON specification into a complete Coq certificate module implementing exactly this chain. Emission is deterministic: the same specification produces byte-identical certificates across machines (verified; see §9). Inadmissible specifications are refused before emission by the same generic gates — there is no per-mechanism or per-window special-casing anywhere in the pipeline.

6. Causal Deviations and the Certified Guarantee

The deviation model is deliberately strong. A **causal deviation strategy** observes the full play history and may react to it arbitrarily, subject only to legality (drawing its actions from the strategy carrier). The guarantee quantifies over *all* such strategies: it is not a claim about

specific named deviations, equilibrium refinements, or behavioral assumptions, but about every history-reactive strategy the formal model can express.

The quantifier structure is exactly:

```
∀ deviation strategy, ∀ player, ∀ ε > 0,
  ∃ horizon H₀, ∀ H ≥ H₀:
    value(deviation, H) ≤ value(compliance, H) + ε
```

The horizon may depend on the deviator; the guarantee's strength is its universality over the class of legal deviators, its exactness (no numerical tolerance), and its provenance (a kernel theorem, not an argument).

7. Three Structural Compression Results

The architecture's central engineering claim — that new structural variation becomes data rather than development — was tested in three experiments under frozen falsification criteria with declared metrics and recorded verdicts; Observations #2 and #3 were prospectively preregistered, while Observation #1 supplies the baseline.

Game variation. Supported games collapsed into a common representation with the two-dimensional sufficient statistic (A, C). Certifying a new game went from a ~1,500-line per-game proof chain to an ~130-line generated certificate; a game excluded by the legacy contract was admitted by the generic family, establishing the generic family as a strict extension of the legacy admissible class.

Mechanism variation. Enforcement mechanisms became a finite descriptor Ω(M) — five rationals and one natural — flowing through one parametric theorem family. Zero special-case compiler branches; constant-size per-mechanism obligations; a deliberately inadmissible mechanism refused by the same generic gate that admits sound ones. The reward-margin portion of the mechanism contract compressed to exactly three scalar inequalities.

Timing variation. The punishment window, fixed at T = 4 through two generations of the system, was shown **not to be structurally necessary**: the certification derivation generalizes to every natural T with *no additional timing contract whatsoever* and no new trajectory branches. `Window4` was vocabulary rather than a necessary semantic premise of this certification derivation — and the kernel adjudicated that. T is now live input data; the same mechanism is certified at T = 2, 3, 4, and 6 through one checker.

Across the three experiments, the generic proof investment *shrank*: 2,485 → 1,595 → 985 lines. The result:

```
games + mechanisms + timing → one reusable
certification architecture
```

The project's own falsification criterion demanded exactly this shape: new structural classes must move complexity upward into generic representations and theorems, leaving only O(1) per-instance obligations — and a capable implementation that instead proliferated cases would have been recorded as a failure. The record describes these as **three structurally distinct compression demonstrations under frozen criteria**.

8. Why the Certifying-Compiler Architecture Matters

An ordinary game-theoretic solver maps input → computed answer. This architecture maps

```
input → decision + formal conditions
      + theorem-backed meaning of acceptance
```

The output is not "the mechanism appears stable." It is: *this exact strategic specification satisfies the formally defined admissibility contract and certificate inequalities, and formal certification establishes this specified causal-deviation guarantee* — with the conditions themselves part of the deliverable, auditable by the customer or a third party.

Economically, the compression results are the scaling statement: a system requiring special-case proof and code per customer scales linearly at best; a platform in which new customer models are *data* flowing through a generic theorem family compounds. The three experiments are the evidence that this system is the second kind.

9. Reproducibility and Trust Boundary

The library is 99 Coq units as listed in its build manifest (`_CoqProject`); a fresh build produces 100 compiled modules — the 99 listed units plus the API smoke test compiled against them. It is built with Coq 8.18.0 under a strict discipline: no `Axiom`, no `Admitted`, no `Parameter`. Kernel verification (`coqchk`) over the full transitive closure of the public API reports `Axioms: <none>`, with no type-in-type, no unsafe (co)fixpoints, and no assumed positivity.

The replication record, reproduced end-to-end on separate hardware from the shipped package:

- Manifest verification: 109/109 files `sha256sum -c` OK.
- Full library build from the manifest-listed sources alone:

`make` exit 0; `coqchk` → `Axioms: <none>`; smoke test green.

- **Generator determinism:** the identical specification fed to

the certifying compiler on two machines produced **byte-identical** certificate modules (SHA-256 agreement), each locally compiled and kernel-audited.

The trust boundary is explicit. Public: the decision surface — the admissibility contract, the certificate inequalities, exact arithmetic, frozen examples, and the guarantee description. Private: the theorem library, the proofs, the certificate generator, and the certifying compiler. The public demonstrator faithfully reproduces what is *decided*; the mechanized development is what makes the guarantee *true*.

10. The Public Demonstrator

A single-file Python program (Apache-2.0, standard library only) lets a visitor select frozen examples or supply their own specification, and see: the admissibility contract evaluated obligation-by-obligation, the certificate clauses at their (T, q), and — on acceptance — the precise guarantee formal certification establishes. Frozen examples span all three outcomes: acceptances at T = 2, 3, 4, 6 (each backed by an actual kernel-checked certificate and marked `KERNEL WITNESS: VERIFIED`); an inadmissible mechanism refused at a discovered margin; and an admissible-but-uncertifiable point at heavy discounting. A demonstrator acceptance is explicitly *not* a kernel certificate; the program says so itself, and the `KERNEL WITNESS` status is bound inside the program to a registry of certified configuration fingerprints and certificate IDs — it can never be asserted by an input specification. The product path is:

```
TRY THE DECISION SURFACE → REQUEST KERNEL CERTIFICATION
```

11. Potential Application Classes

The immediate applications are not "solve arbitrary game theory." Candidate application classes include settings whose strategic structure can be represented within the supported certification model, where an institution must demonstrate that a **specified incentive mechanism possesses a formally defined strategic property**: protocol incentive mechanisms; distributed-system participation rules; marketplace incentive policies; contractual reward/penalty structures; institutional enforcement mechanisms; automated governance rules; multi-agent coordination systems; security mechanisms where deviation incentives matter; machine-to-machine economic protocols.

The commercial primitive is:

```
strategic mechanism → auditable certificate of a
formally specified property
```

— a strategic assurance system, not game-theory software.

12. Current Scope and Limitations

The system does **not** claim: arbitrary N-player certification; arbitrary stochastic games; imperfect- information games; general Nash or sequential-equilibrium computation; continuous strategy spaces; or that UCF/GUTT's relational ontology has thereby been proven.

The completed result is: **a formally defined two-player mediated repeated-game certification architecture whose game, enforcement mechanism, and punishment timing are parametrically variable within its supported class.** Every claim in this paper is scoped to that class, and every acceptance names the exact conditions checked.

13. UCF/GUTT Methodological Significance

This system is a worked realization of the broader UCF/GUTT methodology: relation-first canonical construction, contracts demanded by proofs rather than posited in advance, and a proof kernel as the arbiter of which apparent constraints are necessary premises of a derivation and which are vocabulary. The timing result is the sharpest instance: a constraint carried through two system generations dissolved the moment the kernel was asked whether it was necessary. The paper deliberately does not argue the wider framework; the artifact stands on its own, and its relationship to UCF/GUTT is that it was *built by* the framework's discipline — including frozen falsification criteria under which the structural-compression claim could fail; Observations #2 and #3 were prospectively preregistered, with Observation #1 supplying the baseline.

14. Research Roadmap

The recorded claim-graduation ladder governs what comes next. **Level 1 (established):** one architecture compresses three kinds of variation. **Level 2 (preregistered):** N-player reconstruction — whether the *method* can construct another compressive architecture when the two-player vocabulary fails; success is the pattern re-emerging from new objects, not old theorems generalizing. **Level 3 (to be preregistered):** a genuinely unrelated domain —

whether the methodology itself is domain-transferable, under fairness conditions fixed before any candidate is chosen. Each level's failure is survivable and bounds the claim; each pass is strictly more consequential.

15. Technical Appendix

The capstone theorem (Coq identifier: `tg_checked_causal_deviation_secure_eps`; Coq 8.18; `coqchk`: `Axioms: <none>`): for every window T : nat, mechanism M, game core G, discount q, and legal causal deviation strategy — given `MechanismFeasible M`, `GamePayoffFeasible_M M G`, `CoopRewardFloor_M M G`, `MechRewardMargins M`, and `parametric_checkT T M G q = true` — for every player i < 2 and every ε > 0 there exists H_0 such that for all horizons H ≥ H_0, the mediated finite expected value of the deviation strategy is at most that of compliance plus ε.

Generalization tower (kernel-checked): the original sealed checker equals the mechanism-parametric checker at the original mechanism, which equals the timing-generic checker at (that mechanism, T = 4) — three system generations, each provably a slice of the next.

Verification record (frozen artifacts). Release v0.5.0: `15e49cea…`. Replication package: `aac787b4…` (manifest `9bab434a…`, 109 entries; second-machine: 109/109 OK, build exit 0, `Axioms: <none>`). Cross-machine deterministic certificate: `c8a8da3b4b0879ab`. Public demonstrator: `eb06f6c4…`; README: `e4f43e61…`. Governing doctrine (verdicts, preregistrations, ladder): `ae283f7b…`.